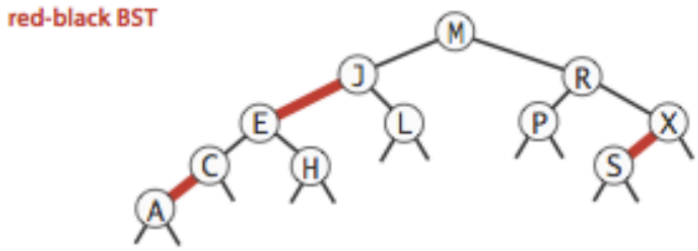
Java 11 - Red-black BST & 2-3 Tree - Detyra të zgjidhura

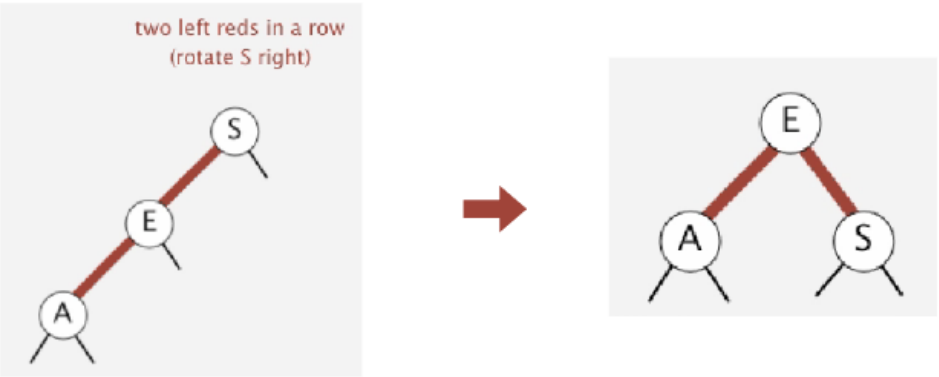
Lënda: Hyrje në Algoritme



- **Çdo herë kur insertohet një element i ri në pemë, linku i tij është i kuq.**
- **Red-black BST vetbalancohen gjatë insertimit të elementeve**

Rregullat:

1) **Dy linka të kuq të lidhur në seri njëri pas tjetrit nuk lejohet të ekzistojnë, prandaj root (rrënja) rrotullohet djathtas**



2) **Një prind nuk mund t'i ketë dy fëmijë me linka të kuq, në këtë rast ua ndërrojmë linkave ngjyrat (flip colors)**





3) **Asnjë link i kuq nuk mund të anohet në anën djathtë, prandaj root (rrënja) rrotullohet majtas**



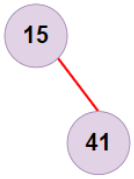
Detyra 1: Nga vargu **A = {15,41,32,10,19,22,67,30,12,48}**

a) Të konstruktohet Red and Black BST
b) Trungu i fituar të kthehet në 2-3 Tree

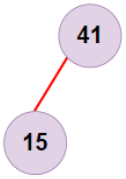
1) insert(15);



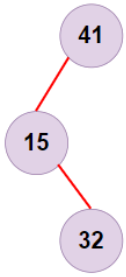
2) insert(41);



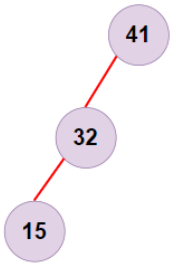
2.1) rotate left 15



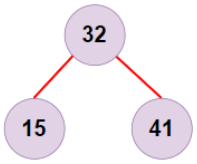
3) insert(32);



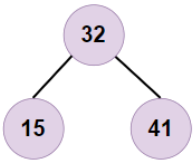
3.1) rotate left 15



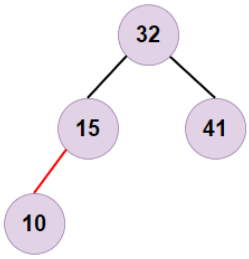
3.2) rotate right 41



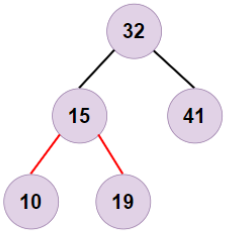
3.3) flip colors



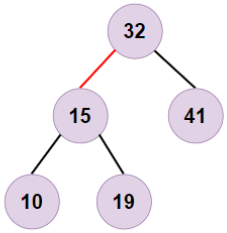
4) insert(10);



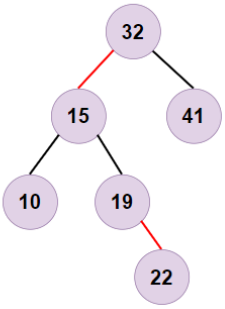
5) insert(19);



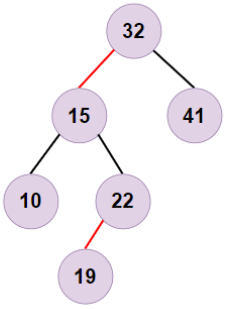
5.1) flip colors



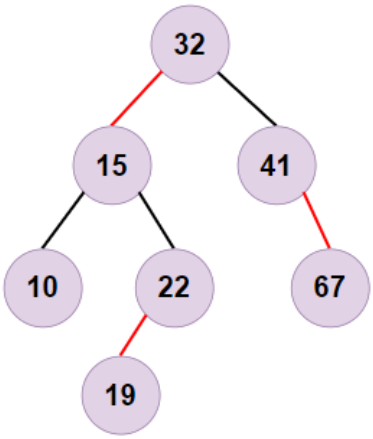
6) insert(22);



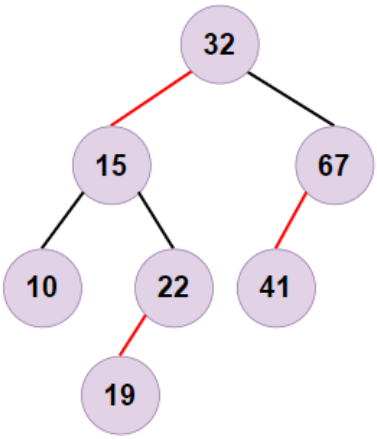
6.1) rotate left 19



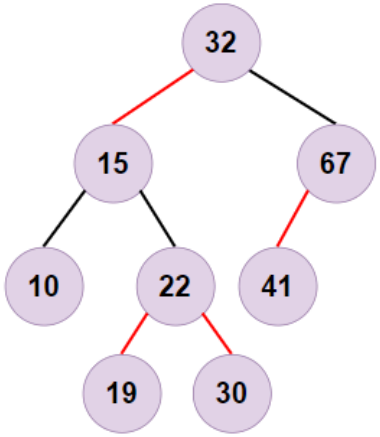
7) insert(67);



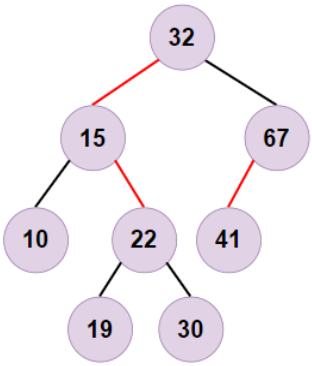
7.1) rotate left 41



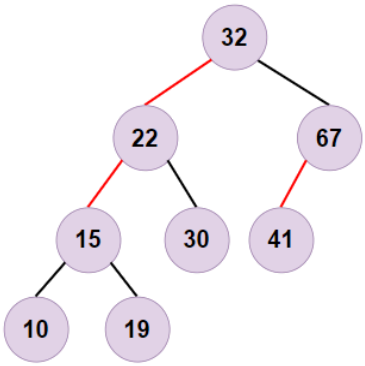
8) insert(30);



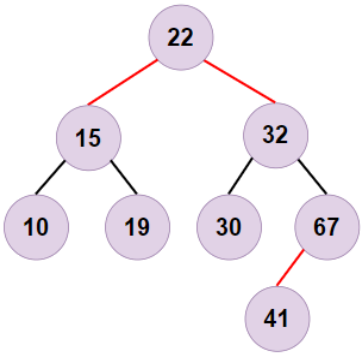
8.1) flip colors



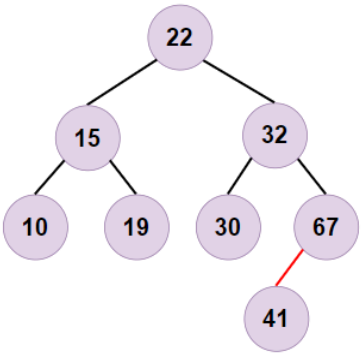
8.2) rotate left 15



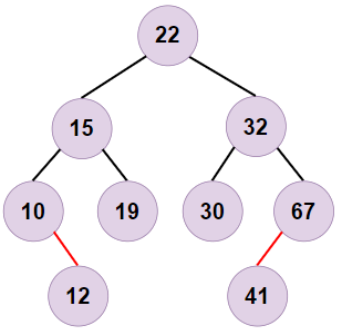
8.3) rotate right 32



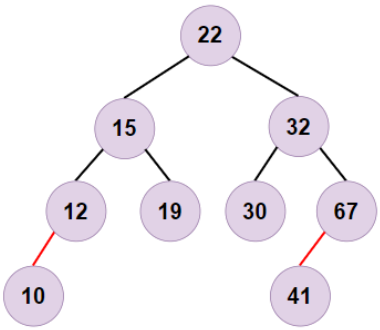
8.4) flip colors



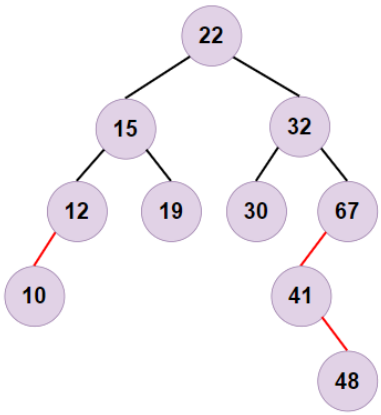
9) insert(12);



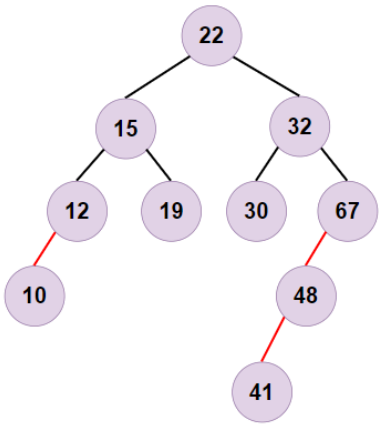
9.1) rotate left 10



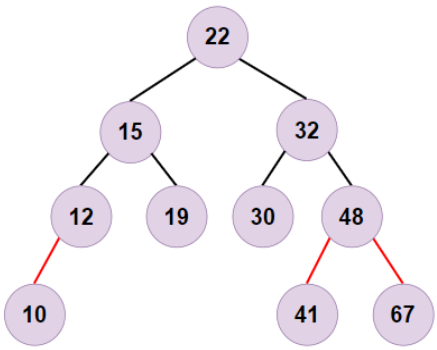
10) insert(48);



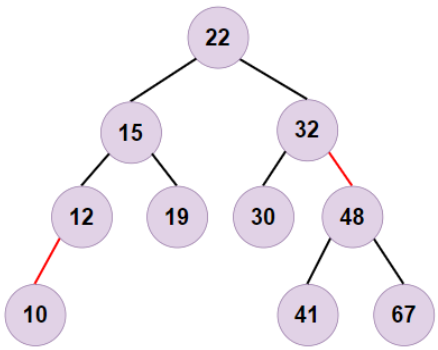
10.1) rotate left 41



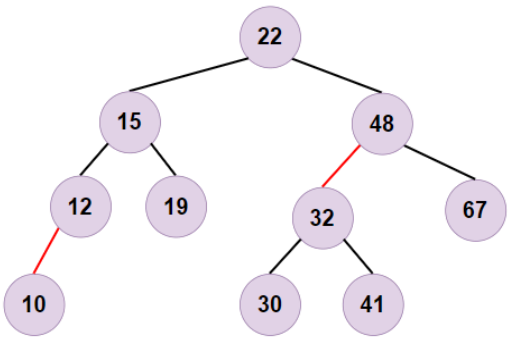
10.2) rotate right 67



10.3) flip colors



10.4) rotate left 32



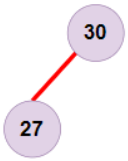
Detyra 2: Nga vargu **A = {30, 27, 20, 25, 29, 28, 26, 40, 52, 17}**

c) Të konstruktohet Red and Black BST
d) Trungu i fituar të kthehet në 2-3 Tree

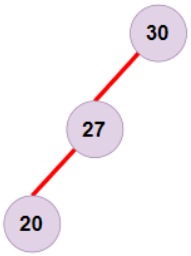
1) insert(30);



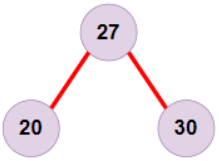
2) insert(27);



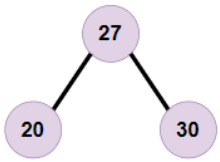
3) insert(20);



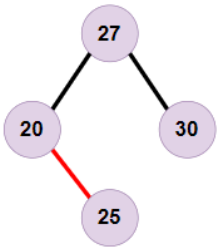
3.1) rotate 30 right



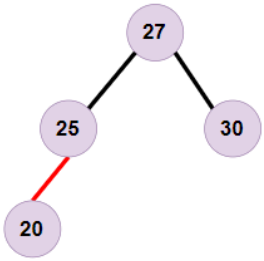
3.2) flip colors



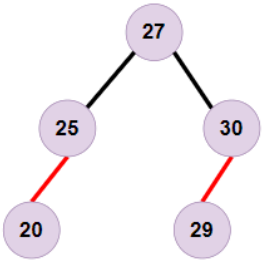
4) insert(25)



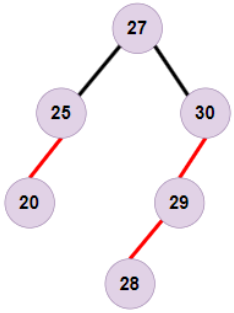
4.1) rotate 20 left



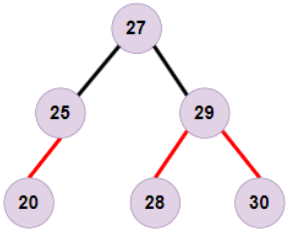
5) insert(29);



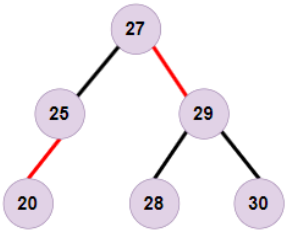
6) insert(28);



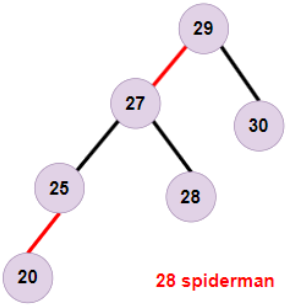
6.1) rotate 30 right



6.2) flip colors

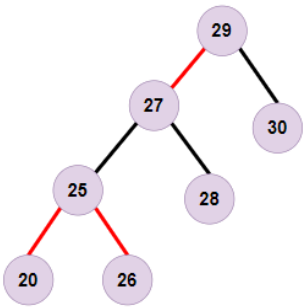


6.3) rotate 27 left

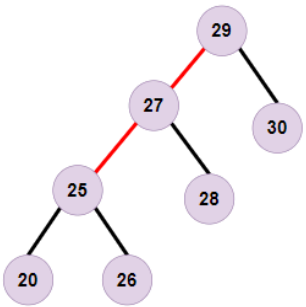


28 spiderman

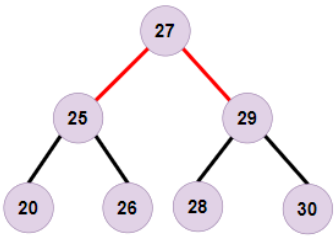
7) insert(26);



7.1) flip colors

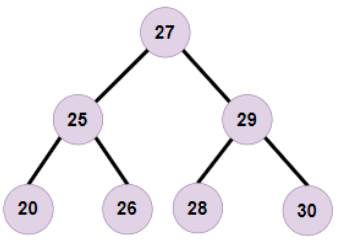


7.2) rotate 29 right

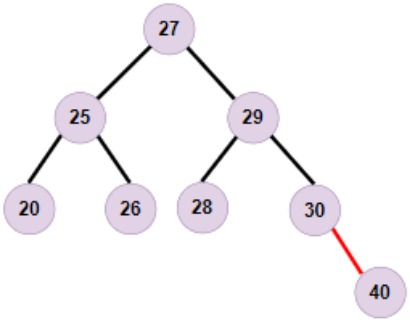


28 spiderman

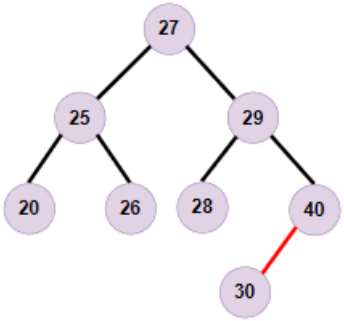
7.3) flip colors



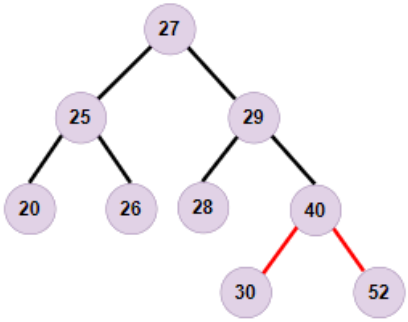
8) insert(40);



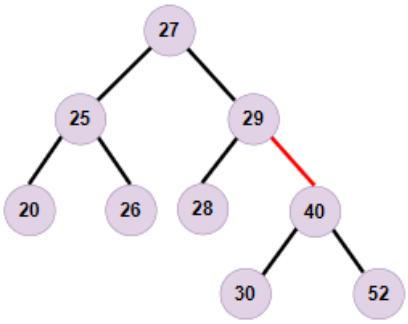
8.1) rotate 30 left



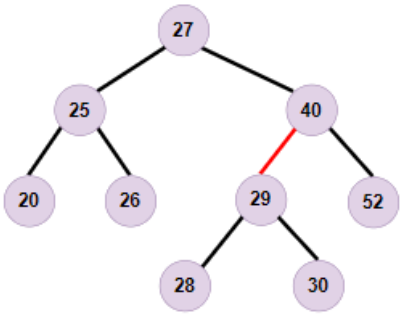
9) insert(52);



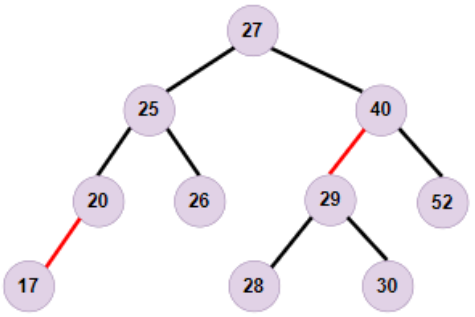
9.1) flip colors



9.2) rotate 29 left

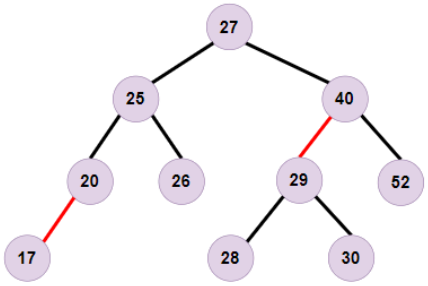


10) insert(17);

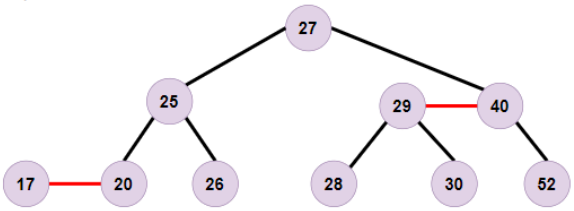


30 spiderman

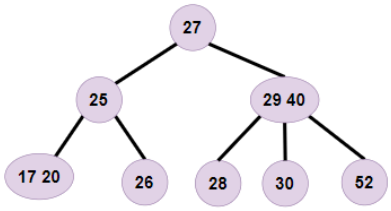
b)



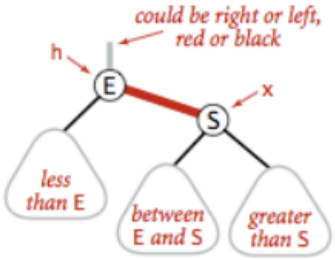
1)



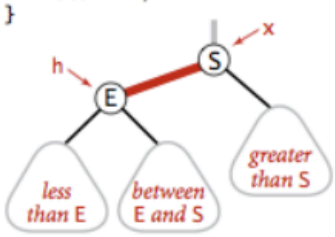
2)



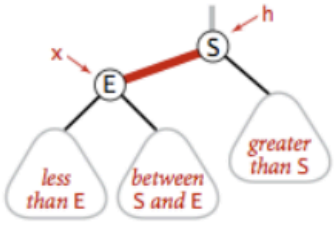
Metodat për rotate left, rotate right dhe flip colors:



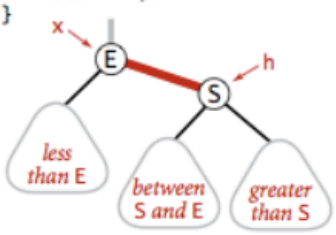

```
Node rotateLeft(Node h)
{
   Node x = h.right;
   h.right = x.left;
   x.left = h;
   x.color = h.color;
   h.color = RED;
   x.N = h.N;
   h.N = 1 + size(h.left)
           + size(h.right);
   return x;
}
```



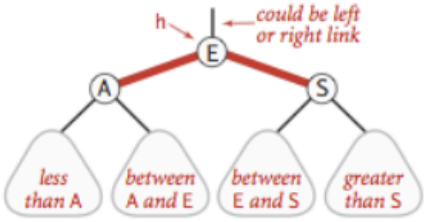
Left rotate (right link of h)



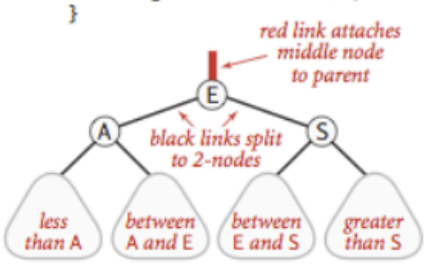

```
Node rotateRight(Node h)
{
   Node x = h.left;
   h.left = x.right;
   x.right = h;
   x.color = h.color;
   h.color = RED;
   x.N = h.N;
   h.N = 1 + size(h.left)
           + size(h.right);
   return x;
}
```



Right rotate (left link of h)

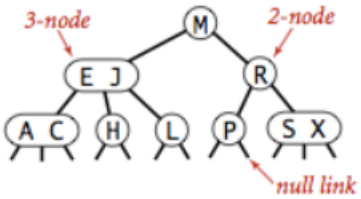



```
void flipColors(Node h)
{
   h.color = RED;
   h.left.color = BLACK;
   h.right.color = BLACK;
}
```



Flipping colors to split a 4-node

2-3 tree

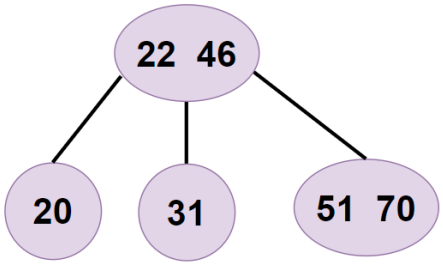


Anatomy of a 2-3 search tree

- **1 key me 2 nodes: në të majtë ; ka key/keys më të vogël**
  në të djathtë; ka key/keys më të mëdhenjë

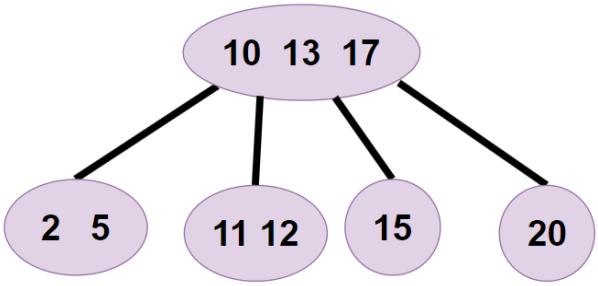
- **2 keys me 3 nodes: në të majtë ; ka key/keys më të vogël**
  në të djathtë; ka key/keys më të mëdhenjë
  në mes; ka key/keys me madhësi ndërmjet 2 keys që ndodhen në node

p.sh:



- **3 keys me 4 nodes: lejohet vetëm përkohësisht.**

p.sh:



- **Pema rritet gjithmonë nga posht - lart.**
- **Të gjitha null links janë të baraz larguara nga rrënja e pemës.**
- **Insertimi: Elementet insertohen në nivelin më të ulët të drurit.**

Detyra 1: Të konstruktohet 2-3 Tree nëse është dhënë vargu
A = {15,41,32,10,19,22,67,30,12,48}

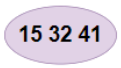
1) insert(15);



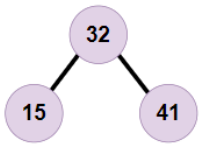
2) insert(41);



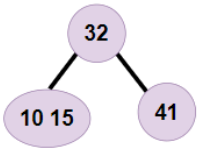
3) insert(32);



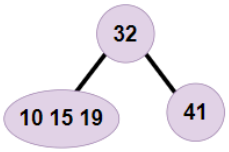
3.1) split 4- node
move 32 to parent



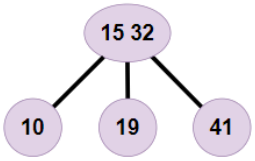
4) insert(10);



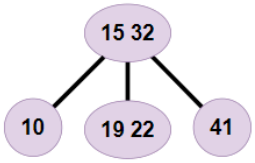
5) insert(19);



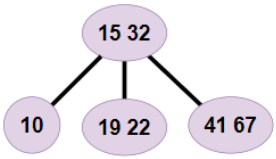
5.1) split 4- node
move 15 to parent



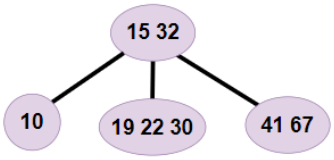
6) insert(22);



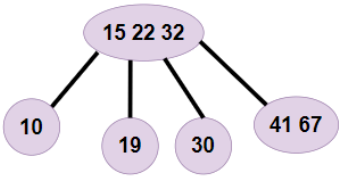
7) insert(67);



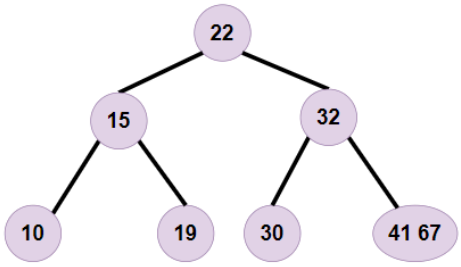
8) insert(30);



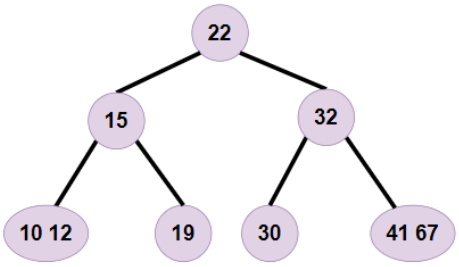
8.1) split 4- node
move 22 to parent



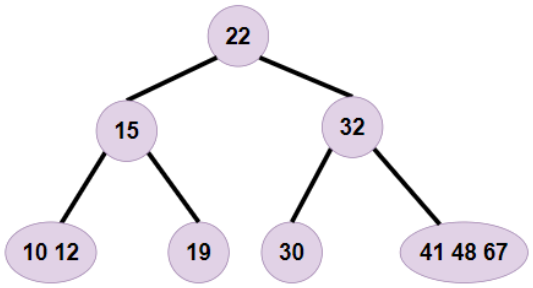
8.2) split 4- node
move 22 to parent



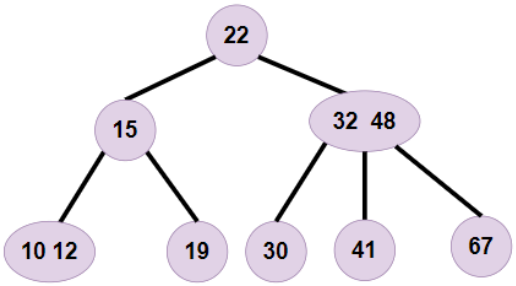
9) insert(12)



10) insert(48)



10.1) split 4- node
move 48 to parent



Detyra 2: Të konstruktohet 2-3 Tree nëse është dhënë vargu
A = {10, 15, 2, 12, 5, 13, 20, 11, 17}

1) insert(10);



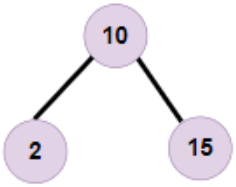
2) insert(15);



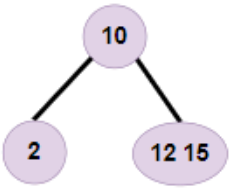
3) insert(2);



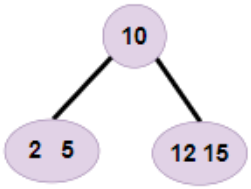
3.1) 5.1) split 4- node move 10 to parent



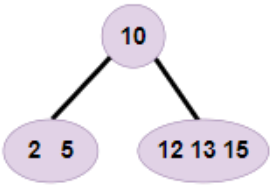
4) insert(12);



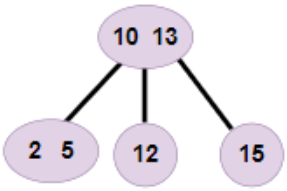
5) insert(5);



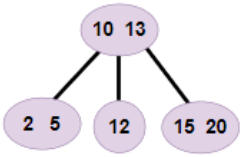
6) insert(13);



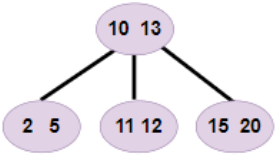
6.1) split 4- node move 13 to parent



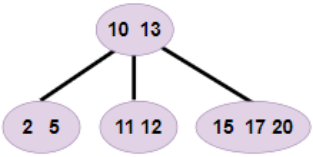
7) insert(20);



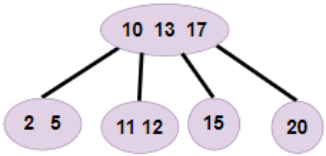
8) insert(11);



9) insert(17);



9.1) split 4- node move 17 to parent



9.2) split 4- node move 13 to parent

